

4.a) and 4.b) Dynamics as Mixed Invariant on $SE_2(2)$ and Adding Wind (World-Frame):

Question:

Test group affine property for fixed wing aircraft dynamics with and without wind

Solution:

Code snippet 5 shows the verification of group affine property of the $SE_2(2)$ windy fixed wing aircraft dynamics

Code Snippet 1: $SE_2(2)$ Fixed Wing Aircraft Group Affine Test

```

1 import pytest as pt
2 import numpy as np
3 from Code.S02.S02_maps import so2_wedge
4 from Code.SE22.SE22_maps import se22_compose, se22_inverse, se22_exp, se22_log, se22_ee,
   se22_wedge, se22_Ad, se22_ad
5
6 def test_se22_group_affine():
7     """
8     Test that group affine property holds for random SE2 elements with mixed invariant
9     dynamics  $f_u(X) = xi\_R\_wedge X + X xi\_L\_wedge$ 
10     $f_u(XY) = f_u(X) Y + X f_u(Y) - X f_u(I) Y$ 
11    """
12    n_samples = 100
13
14    rng = np.random.default_rng()
15    xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
16    xi_Y_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
17    a = np.reshape(rng.uniform(-10, 10, 2 * n_samples), (2, n_samples))
18    omega = rng.uniform(-10, 10, n_samples)
19
20    checks = np.zeros([n_samples, 1])
21    for i in range(n_samples):
22        X_random = se22_exp(xi_X_random[:, i])
23        Y_random = se22_exp(xi_Y_random[:, i])
24
25        M = np.zeros((4, 4))
26        N = np.block([[so2_wedge(omega[i]), a[:, i].reshape((2, 1)), np.zeros((2, 1))], [np.
27        zeros((2, 4))]])
28        C = np.block([np.zeros((4, 3)), np.array((0, 0, 1, 0)).reshape((4, 1))])
29        xi_R_wedge = M - C
30        xi_L_wedge = N + C
31
32        lhs = xi_R_wedge @ X_random @ Y_random + X_random @ Y_random @ xi_L_wedge # f_u(XY)
33        rhs = (xi_R_wedge @ X_random + X_random @ xi_L_wedge) @ Y_random + X_random @ (
34        xi_R_wedge @ Y_random + Y_random @ xi_L_wedge) - X_random @ (xi_R_wedge + xi_L_wedge) @
35        Y_random # f_u(X) Y + X f_u(Y) - X f_u(I) Y
36
37        checks[i] = np.all(lhs == pt.approx(rhs))
38
39    assert np.all(checks)
40
41 def test_se22_group_affine_wind():
42     """
43     Test that group affine property holds for random SE2 elements with mixed invariant
44     dynamics with wind  $f_u(X) = xi\_R\_wedge X + X xi\_L\_wedge$ 
45     $f_u(XY) = f_u(X) Y + X f_u(Y) - X f_u(I) Y$ 
46    """
47    n_samples = 100
48
49    rng = np.random.default_rng()
50    xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
51    xi_Y_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
52    a = np.reshape(rng.uniform(-10, 10, 2 * n_samples), (2, n_samples))
53    omega = rng.uniform(-10, 10, n_samples)
54
55    w_a = np.array((3, -2)).reshape((2, 1)) # [m / s2]

```

```

51
52     checks = np.zeros([n_samples, 1])
53     for i in range(n_samples):
54         X_random = se22_exp(xi_X_random[:, i])
55         Y_random = se22_exp(xi_Y_random[:, i])
56
57         M = np.block([np.zeros((4, 2)), np.vstack((w_a, np.zeros((2, 1))))], np.zeros((4, 1))
58         ])
59         N = np.block([[so2_wedge(omega[i]), a[:, i].reshape((2, 1)), np.zeros((2, 1))], [np.
60         zeros((2, 4))]])
61         C = np.block([np.zeros((4, 3)), np.array((0, 0, 1, 0)).reshape((4, 1))])
62         xi_R_wedge = M - C
63         xi_L_wedge = N + C
64
65         lhs = xi_R_wedge @ X_random @ Y_random + X_random @ Y_random @ xi_L_wedge # f_u(XY)
66         rhs = (xi_R_wedge @ X_random + X_random @ xi_L_wedge) @ Y_random + X_random @ (
67         xi_R_wedge @ Y_random + Y_random @ xi_L_wedge) - X_random @ (xi_R_wedge + xi_L_wedge) @
68         Y_random # f_u(X) Y + X f_u(Y) - X f_u(I) Y
69
70         checks[i] = np.all(lhs == pt.approx(rhs))
71
72     assert np.all(checks)

```

Figure 1: Windless and Windy Fixed Wing Aircraft Group Affine Test Passing

```

(.venv) PS C:\Users\thatf\OneDrive\Documents\Purdue Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods> pytest Tests\SE22\test_SE22_group_affine.py
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\thatf\OneDrive\Documents\Purdue Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods
collected 2 items

Tests\SE22\test_SE22_group_affine.py ..                                     [100%]

===== 2 passed in 0.17s =====

```

4.c) 2D INS Simulator

Code Snippet 2: 2D INS Simulator Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from Code.SE22.SE22_maps import se22_compose, se22_exp, se22_log, se22_wedge, se22_vee,
4     se22_inverse
5 from Code.SE22.SE22_integrators import se22_Lie_group_integrator
6
7 def turning_radius(velocity, desired_turn_rate):
8     g = 9.81 # [m / s^2]
9     turn_radius = velocity ** 2 / (g * np.tan(desired_turn_rate))
10
11     return turn_radius
12
13 ## 1b.2) Propagate waypoints
14 X_0 = np.eye(4)
15
16 # Segment info
17 dt = 0.02 # [s]
18 N_seg = 4
19 a = np.array([[2, 0, 0, -1],
20               [0, 0, 0, 0]]) # [m / s]
21 omega = np.array([0, 0, 0.3, 0]).reshape([1, 4]) # [rad / s]
22 T = np.array([5, 5, np.round(np.pi / 0.3 / dt) * dt, 5]) # [s]
23
24 xi_k = np.vstack((a, np.zeros_like(a), omega)).reshape([5, 1, -1])
25
26 # Simulation loop
27 X_k = np.zeros([4, 4, N_seg + 1]) # Waypoint group elements
28 X_k[:, :, 0] = X_0

```

```

29
30 T_seg = np.cumsum(T)
31 time = np.arange(0, T_seg[-1], dt)
32 k_seg = np.round(T / dt).astype(int)
33 k_seg_total = sum(k_seg) + 1
34 X_seg = np.zeros([4, 4, k_seg_total])
35 X_seg[:, :, 0] = X_0
36 xi_seg = np.hstack((np.matlib.repmat(xi_k[:, :, 0], 1, k_seg[0]),
37                      np.matlib.repmat(xi_k[:, :, 1], 1, k_seg[1]),
38                      np.matlib.repmat(xi_k[:, :, 2], 1, k_seg[2]),
39                      np.matlib.repmat(xi_k[:, :, 3], 1, k_seg[3])))
40
41 for k in range(N_seg):
42     xi = xi_k[:, :, k]
43     if xi[4] != 0: # Add centripital acceleration
44         rvec = X_k[0:2, 3, k].reshape([2, 1])
45         r = np.linalg.norm(rvec)
46         vvec = X_k[0:2, 2, k].reshape([2, 1])
47         v = np.linalg.norm(vvec)
48         vhat = vvec / v
49         omega = xi[4]
50         r_turn = turning_radius(v, omega)
51         a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ vhat
52         xi[0:2] = xi[0:2] + a_centrip
53     X_k[:, :, k + 1] = se22_Lie_group_integrator(X_k[:, :, k], xi, T[k])
54     xi_kc = xi_k[:, :, k]
55     X_kc = X_k[:, :, k]
56     X_kp1 = X_k[:, :, k + 1]
57
58     hiii = 1
59
60 for s in range(k_seg_total - 1):
61     xi = xi_seg[:, s].reshape([5, 1])
62     if xi[4] != 0: # Add centripital acceleration
63         rvec = X_seg[0:2, 3, s].reshape([2, 1])
64         r = np.linalg.norm(rvec)
65         vvec = X_seg[0:2, 2, s].reshape([2, 1])
66         v = np.linalg.norm(vvec)
67         vhat = vvec / v
68         omega = xi[4]
69         r_turn = turning_radius(v, omega)
70         a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ vhat
71         xi[0:2] = xi[0:2] + a_centrip
72     X_seg[:, :, s + 1] = se22_Lie_group_integrator(X_seg[:, :, s], xi, dt)
73
74 # Plot
75 # Trajectories
76 plt.plot(X_seg[0, 3, :], X_seg[1, 3, :])
77 # plt.quiver(X_k[0, 3, :], X_k[1, 3, :], X_k[0, 2, :], X_k[1, 2, :])
78 # for k in range(N_seg):
79 #     plt.text(X_k[0, 3, k], X_k[1, 3, k], f"Segment {k}", fontsize=10,
80 #             bbox=dict(facecolor='red', alpha=0.5))
81 plt.xlabel("X Position [m]")
82 plt.ylabel("Y Position [m]")
83 plt.title("Flight Profile in (x, y) Plane")
84 plt.legend()
85 plt.grid()
86 plt.gca().set_aspect('equal', adjustable='box')
87 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_traj.png")
88 plt.show()
89
90 vel_mag = np.linalg.vector_norm(X_seg[0:2, 2, :], axis = 0)
91 plt.plot(time, vel_mag[0:-1])
92 plt.grid()
93 plt.title("Velocity vs Time")
94 plt.xlabel("Time [s]")
95 plt.ylabel("Velocity [m / s]")
96 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_vel.png")

```

```

97 plt.show()
98
99 heading = np.zeros([k_seg_total - 1, 1])
100 for k in range(k_seg_total - 1):
101     xi_recover = se22_log(X_seg[:, :, k])
102     heading[k] = xi_recover[4]
103 plt.plot(time, np.rad2deg(heading))
104 plt.grid()
105 plt.title("Heading vs Time")
106 plt.xlabel("Time [s]")
107 plt.ylabel("Heading [deg]")
108 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_head.png")
109 plt.show()
110
111 plt.plot(time, X_seg[0, 3, 0:-1])
112 plt.plot(time, X_seg[1, 3, 0:-1])
113 plt.grid()
114 plt.title("Position vs Time")
115 plt.xlabel("Time")
116 plt.ylabel("Position")
117 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_pos.png")
118 plt.show()
119
120 # Simulate spiral
121 dt_spiral = 0.01 # [s]
122 tf = 10 # [s]
123 N_t = np.round(tf / dt_spiral).astype(int)
124 X_k_spiral = np.zeros([4, 4, N_t + 1])
125 omega_spiral = 0.5 # [rad / s]
126 a_spiral = np.array([[1], [0]])
127 xi_spiral = np.block([[a_spiral], [np.zeros((2, 1))], [omega_spiral]]).reshape([5, 1])
128 X_k_spiral[:, :, 0] = X_0
129
130 for k in range(N_t):
131     xi = xi_spiral
132     if xi[4] != 0: # Add centripital acceleration
133         rvec = X_k_spiral[0:2, 3, k].reshape([2, 1])
134         r = np.linalg.norm(rvec)
135         vvec = X_k_spiral[0:2, 2, k].reshape([2, 1])
136         v = np.linalg.norm(vvec)
137         if v < 1e-5:
138             vhat = np.zeros([2, 1])
139             r_turn = 1
140         else:
141             vhat = vvec / v
142             omega = xi[4]
143             r_turn = turning_radius(v, omega)
144             a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ vhat
145             xi[0:2] = xi[0:2] + a_centrip
146             X_k_spiral[:, :, k + 1] = se22_Lie_group_integrator(X_k_spiral[:, :, k], xi, dt_spiral)
147
148 plt.plot(X_k_spiral[0, 3, :], X_k_spiral[1, 3, :], label = "Line")
149 v_ind = np.round(np.linspace(0, N_t, 10)).astype(int)
150 plt.quiver(X_k_spiral[0, 3, v_ind], X_k_spiral[1, 3, v_ind], X_k_spiral[0, 2, v_ind],
151            X_k_spiral[1, 2, v_ind])
152 plt.xlabel("X [m]")
153 plt.ylabel("Y [m]")
154 plt.title("Spiral")
155 plt.legend()
156 plt.gca().set_aspect('equal', adjustable='box')
157 plt.grid()
158 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_spiral.png")
159 plt.show()
160
161 vel_mag = np.linalg.vector_norm(X_k_spiral[[0, 1], 2, :], axis = 0)
162 plt.plot(np.linspace(0, tf, N_t + 1), vel_mag)
163 plt.grid()
164 plt.title("Velocity vs Time for Spiral")

```

```

164 plt.xlabel("Time [s]")
165 plt.xlabel("Velocity [m / s]")
166 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_spiralvel.png")
167 plt.show()
168
169 hi = 1

```

Figure 2: Plots of trajectory

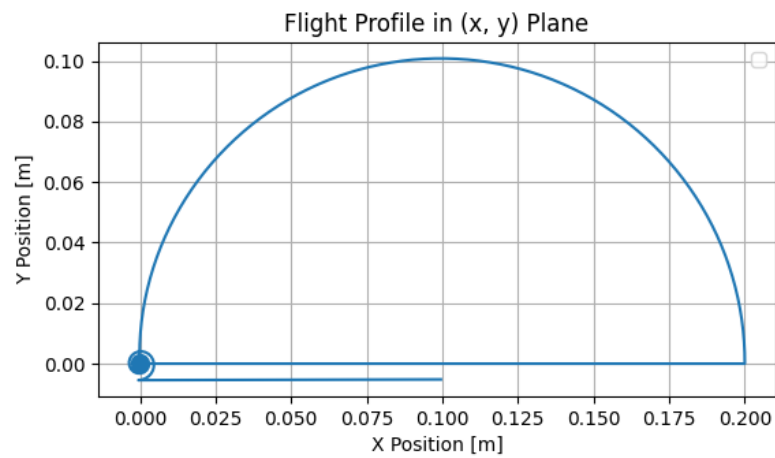


Figure 3: Plots of velocity

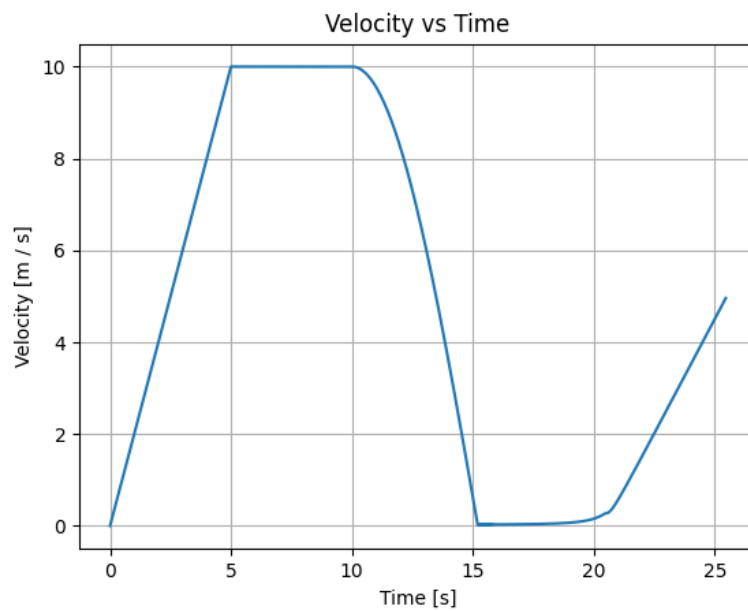
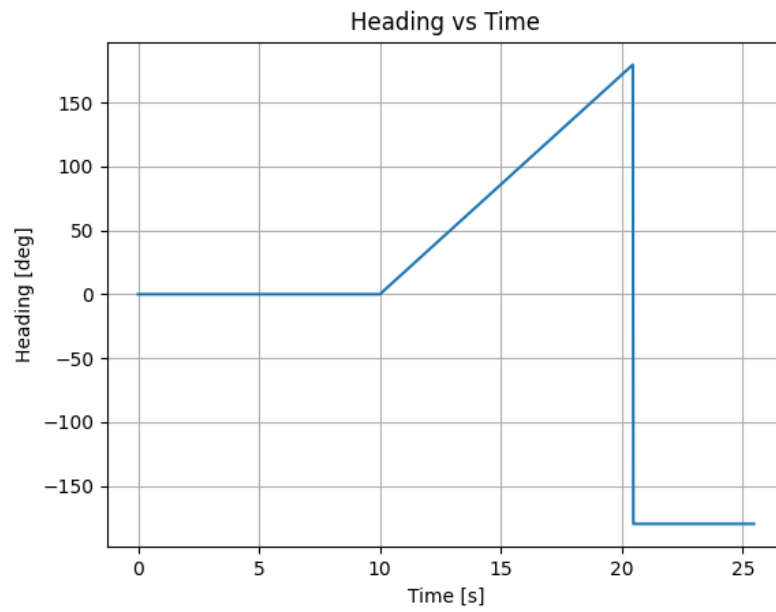


Figure 4: Plots of heading



Spiral test

Figure 5: Plots of Spiral

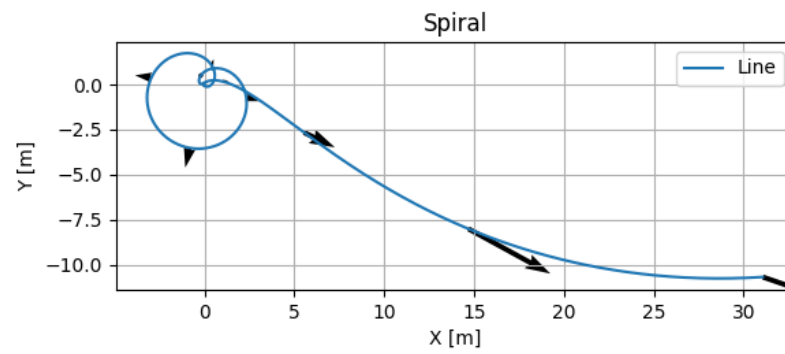


Figure 6: Plots of Spiral velocity

